

Лабораторная работа №1

Простое консольное приложение

ЗАДАНИЕ:

1. Создать консольное приложение. Название проекта: НомерГруппы_Фамилия.
2. Ознакомиться со структурой проекта.
3. Найти файл *.csproj. Найдите в нем указание на целевую платформу и тип приложения.
4. Добавьте в проект NuGet пакет AutoMapper. Найдите ссылку на установленный пакет в файле *.csproj.
5. Написать программу, которая выводит в консоль частное от деления чисел, введенных с клавиатуры. Введенные данные и результат сохранять в отдельных переменных.
6. Запустить проект в двух режимах: с отладкой и без отладки. Ввести сначала корректные, а затем заведомо неправильные значения чисел (строка вместо числа, ноль в качестве делителя). Сравнить результат в режиме отладки и выпуска.
7. Найти скомпилированные файлы (для режима отладки и режима выпуска)
8. Поставить точки останова. Запустить проект в двух режимах: с отладкой и без отладки. Сравнить результат. Прочитать в отладчике значения переменных. Провести пошаговое выполнение программы.

Составить отчет с пояснениями и скриншотами по каждому пункту задания.

Лабораторная работа №2

Управление потоком выполнения программы

Примечание: номер варианта выбирать согласно номеру зачетной книжки. Если номер больше максимального номера варианта, продолжать отсчет, начиная с первого варианта.

Примечание: Для каждого задания написать меню с двумя пунктами: Продолжить и Закончить. Использовать любой оператор цикла и оператор switch.

ЗАДАНИЕ 1:

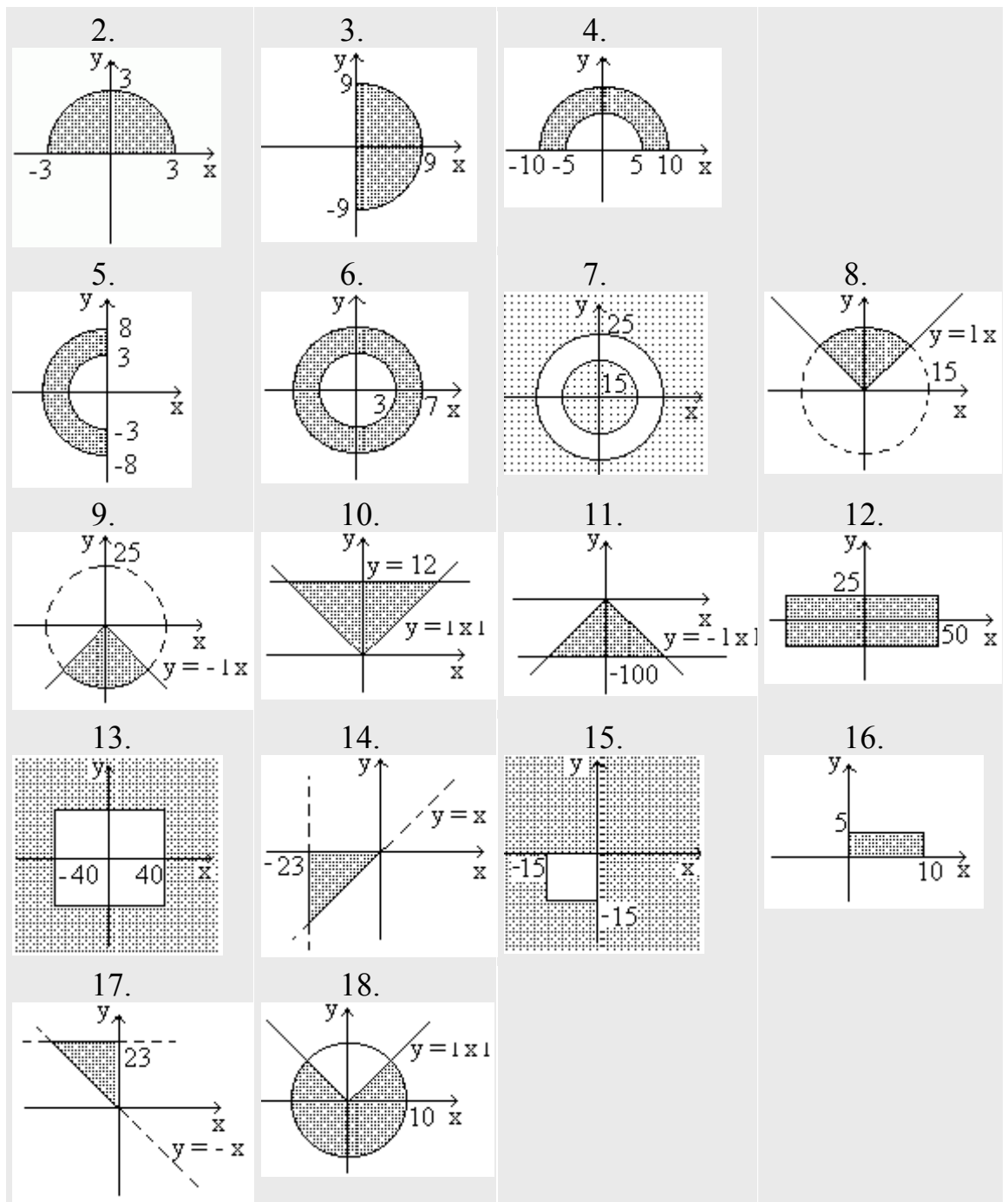
Написать программу, которая определяет:

1. является ли заданное целое число четным;
2. является ли заданное целое число нечетным;
3. если целое число M делится на целое число N , то на экран выводится частное от деления, в противном случае выводится сообщение "М на N нацело не делится";
4. оканчивается ли данное целое число цифрой 7;
5. имеет ли уравнение $ax^2+bx+c=0$ решение, где a , b , c – данные вещественные числа;
6. какая из цифр двухзначного числа больше: первая или вторая;
7. одинаковы ли цифры данного двухзначного числа;
8. является ли сумма цифр двухзначного числа четной;
9. является ли сумма цифр двухзначного числа нечетной;
10. кратна ли трем сумма цифр двухзначного числа;
11. кратна ли числу A сумма цифр двухзначного числа;
12. какая из цифр трехзначного числа больше: первая или последняя;
13. какая из цифр трехзначного числа больше: первая или вторая;
14. какая из цифр трехзначного числа больше: вторая или последняя;
15. все ли цифры трехзначного числа одинаковые;
16. существует ли треугольник с длинами сторон a , b , c ;
17. является ли треугольник с длинами сторон a , b , c прямоугольным;
18. является ли треугольник с длинами сторон a , b , c равнобедренным;
19. является ли треугольник с длинами сторон a , b , c равносторонним.

ЗАДАНИЕ 2:

Дана точка на плоскости с координатами (x, y) . Составить программу, которая выдает одно из сообщений "Да", "Нет", "На границе" в зависимости от того, лежит ли точка внутри заштрихованной области, вне заштрихованной области или на ее границе.

Области задаются графически следующим образом:



Вопросы для самопроверки

1. Как задать бесконечный цикл?
2. Как реализовать ветвление программы с помощью оператора **if-else**?
3. Чем отличается **break** от **continue** в операторе **for**?
4. Как реализовать вложенные циклы **for**?
5. Как в операторе **switch** указать одинаковое действие для нескольких значений **case**?
6. Как в операторе **switch** указать действие в случае, если значение переменной не совпало ни с одним значением **case**?

7. Чем отличается цикл **while** от **do-while**
8. Как выйти из цикла **while(true)** ?
9. Чем отличается тип данных **double** от **decimal**?
10. Как объявить **литерал** типа **float** ?
11. Какое значение будет у переменной **result** :

```
int a = 3; int b = 2;  
var result = a / b;
```

12. Чем отличается ссылочный тип от типа значений (структурного типа)?
13. К какому типу относится тип **int** ?
14. Как преобразовать строку к типу **double**?
15. Чем отличается **Int32.Parse** от **Int32.TryParse**?
16. Можно ли переменной типа **int** присвоить значение типа **double** ?
17. Можно ли переменной типа **double** присвоить значение типа **int** ?

Лабораторная работа №3
Классы математических функций, классы для работы с числами и датами

Цель работы

Научиться описывать простые классы

Время выполнения – 4 академических часа (2 занятия)

Задание к работе

Примечание 1: номер варианта выбирать согласно номеру зачетной книжки. Если номер больше максимального номера варианта, продолжать отсчет, начиная с первого варианта.

Примечание 2: ввод-вывод данных реализовать **Вне** проектируемого класса.

Задание 1. Описать класс (с произвольным именем) в отдельном файле проекта. В созданном классе:

1. Разработать метод $\min(a,b)$ для нахождения минимального из двух чисел. Вычислить с помощью него значение выражения по следующей формуле: $z = \min(x, 2y+x) - \min(7x+2y, y)$.
2. Разработать метод $\max(a,b)$ для нахождения максимального из двух чисел. Вычислить с помощью него значение выражения по следующей формуле: $z = \max(x, 2y+x) - \max(7x+2y, y)$.
3. Разработать метод $f(x)$, который вычисляет значение по следующей формуле: $f(x) = x - \sin x$. Определить, в какой из точек a или b , функция принимает наименьшее значение.
4. Разработать метод $f(x)$, который вычисляет значение по следующей формуле: $f(x) = \cos x - \sin 2x$. Определить, в какой из точек a или b , функция принимает наименьшее значение.
5. Разработать метод $f(n)$, который для заданного натурального числа n находит значение n^2 . Вычислить с помощью него значение выражения $2^2 + 3^2 + 4^2$.
6. Разработать метод $f(n, x)$, который для заданного натурального числа n и вещественного x находит значение выражения x^n/n . Вычислить с помощью данного метода значение выражения $x^2/2 + x^4/4 + x^6/6$.
7. Разработать метод, который нечетное число заменяет на 0 , а четное число уменьшает в два раза. Продемонстрировать работу данного метода на примере.

8. Разработать метод, который число, кратное 2, уменьшает в 2 раза, а остальные числа увеличивает на 3. Продемонстрировать работу данного метода на примере.

9. Разработать метод, который в двузначном числе меняет цифры местами, а остальные числа оставляет без изменения. Продемонстрировать работу данного метода на примере.

10. Разработать метод, который в трехзначном числе меняет местами первую с последней цифрой, а остальные числа оставляет без изменения. Продемонстрировать работу данного метода на примере.

Задание 2. Добавить в проект папку Services. В созданной папке описать класс (с произвольным именем). В созданном классе описать метод, вычисляющий значение функции в соответствии с индивидуальным вариантом. Метод должен вывести в консоль результат выполнения и номер ветки, по которой производилось вычисление.

1. $y = (\ln(1+x^2) + \cos(x+1))e^{k \cdot x}$, где $x = \begin{cases} k \cdot z^3, & \text{при } k < 1 \\ z \cdot (z+1), & \text{при } k \geq 1 \end{cases}$
2. $y = \frac{a \cdot x + b \cdot x \cdot \cos \sqrt{x}}{x + a \cdot b}$, где $x = \begin{cases} \sqrt{a^2 + b^2 \cdot z}, & \text{при } z < a \cdot b \\ \sin^2 z + |a \cdot b \cdot z|, & \text{при } z \geq a \cdot b \end{cases}$
3. $y = -\pi + \cos^2 x^3 + \sin^3 x^2$, где $x = \begin{cases} z/b, & \text{при } z < 1 \\ \sqrt{(z \cdot b)^3}, & \text{при } z \geq 1 \end{cases}$
4. $y = \cos^3 x^2 + \sin^2 x^3$, где $x = \begin{cases} z^3 + 0,2, & \text{при } z < 1 \\ z + \ln z, & \text{при } z \geq 1 \end{cases}$
5. $y = \ln(x+0,5) + (e^x - e^{-x})$, где $x = \begin{cases} -z/3, & \text{при } z < -1 \\ |z|, & \text{при } z \geq -1 \end{cases}$
6. $y = \frac{2}{3} \sin^2 x - \frac{3}{4} \cos^2 x$, где $x = \begin{cases} z, & \text{при } z < 0 \\ \sin z, & \text{при } z \geq 0 \end{cases}$
7. $y = \sin^3(c \cdot x + d^2 + k \cdot x^2)$, где $x = \begin{cases} z^2 - z, & \text{при } z < 0 \\ z^3, & \text{при } z \geq 0 \end{cases}$
8. $y = \ln(x+0,5) + (e^x - e^{-x})$, где $x = \begin{cases} -z/3, & \text{при } z < -1 \\ |z|, & \text{при } z \geq -1 \end{cases}$
9. $y = \frac{1}{\cos x} + \ln \left| \operatorname{tg} \frac{x}{2} \right|$, где $x = \begin{cases} z^b + \left| \frac{b}{2} \right|, & \text{при } z \leq 0 \\ \sqrt{z}, & \text{при } z > 0 \end{cases}$
10. $y = \frac{e^{\sin^3 x} + \ln(x+1)}{\sqrt{x}}$, где $x = \begin{cases} z-1, & \text{при } z \geq 1 \\ z^2 + 1, & \text{при } z < 1 \end{cases}$

$$\begin{array}{ll}
11. y = \frac{2e^{-3x} - 4x^2}{\ln|x| + x}, & \text{где } x = \begin{cases} \frac{1}{z^2 + 2z}, & \text{при } z > 0 \\ 1 - z^3, & \text{при } z \leq 0 \end{cases} \\
12. y = \sin^3(x^2 - 1) + \ln|x| + e^x, & \text{где } x = \begin{cases} z^2 + 5, & \text{при } z \leq 0 \\ \frac{1}{\sqrt{z-1}}, & \text{при } z > 0 \end{cases} \\
13. y = \sin(n \cdot x) + \cos(k \cdot x) + \ln(m \cdot x), & \text{где } x = \begin{cases} e^z + z, & \text{при } z > 1 \\ z^2 + 1, & \text{при } z \leq 1 \end{cases} \\
14. y = \cos 5x + \sin \frac{1}{5}x + e^x, & \text{где } x = \begin{cases} \sqrt{z}, & \text{при } z > 0 \\ (3z^3 - z) - 5, & \text{при } z \leq 0 \end{cases} \\
15. y = x(\sin x + e^{-(x+3)}), & \text{где } x = \begin{cases} -3z, & \text{при } z > 0 \\ z^2, & \text{при } z \leq 0 \end{cases} \\
16. y = \sin^2 x + \cos^5 x^3 + \ln x^{2/5}, & \text{где } x = \begin{cases} 2z + 1, & \text{при } z \geq 0 \\ \ln(z^2 - z), & \text{при } z < 0 \end{cases}
\end{array}$$

Задание 3 В отдельном файле описать класс DateService.

Класс должен предоставлять следующие методы:

Определять день недели произвольной даты, передаваемой методу в качестве **строкового** параметра (например GetDay(string date))

Определять, сколько дней пройдет между текущей датой и датой, передаваемой методу в качестве **числовых** параметров: **день, месяц, год** (например GetDaysSpan(int day, int month, int year);

Вопросы для самопроверки

1. Как узнать длину строки?
2. Как получить предпоследний символ строки?
3. Что такое **конкатенация** строк?
4. Строка – это неизменяемый объект. Как тогда осуществляется замена подстроки методом **Replace** ?
5. Как сравниваются строки?
6. Какие методы существуют для сравнения строк?
7. Что такое «интерполяция строк»?
8. Как убрать лишние пробелы в начале и в конце строки?
9. Что делает со строкой метод **Split** ?
10. Как используется метод **String.Format** ?

11. Как перевести всю строку в верхний/нижний регистр?
12. Как получить объект **DateTime** с требуемой датой?
13. Как получить день недели, который будет через 20 дней после **сегодняшней** даты?
14. Для чего используется тип данных **TimeSpan**?

Лабораторная работа № 4
Конструкторы. Статические члены класса.
Шаблон проектирования Singleton.

Цель работы:

- Изучить работу и назначение конструкторов;
- Освоить во возможности членов класса с модификатором static;
- Ознакомиться с шаблоном проектирования Singleton;

Задание к работе

Примечание: номер варианта выбирать согласно номеру зачетной книжки. Если номер больше максимального номера варианта, продолжать отсчет, начиная с первого варианта.

- Спроектировать классы для выбранной предметной области: класс-контейнер, управляющий контейнеризируемым классом, и контейнеризируемый класс.
- Предусмотреть у классов наличие полей, методов и свойств.
- Предусмотреть перегрузку методов.
- Предусмотреть наличие статических методов.
- Применить к классу-контейнеру шаблон проектирования Singleton.

Индивидуальные задания

1. Предметная область: **АТС-Тариф**. В классе-контейнере хранить информацию об адресе АТС, числе абонентов, абонентской плате (одинаковая для всех направлений - класс Тариф). Реализовать метод для подсчета абонентской платы всех клиентов. Реализовать возможность изменения (увеличения и уменьшения) тарифа.

2. Предметная область: **Вокзал-Билет**. В классе хранить информацию о наименовании станции, стоимости билета (одинаковая для всех направлений - класс Билет), число мест, число проданных билетов. Реализовать метод для подсчета общей стоимости всех не проданных билетов. Реализовать возможность изменения (увеличения и уменьшения) стоимости билета.

3. Предметная область: **ЖЭС-Тариф**. В классе хранить информацию о районе, к которому принадлежит ЖЭС, номер ЖЭС, число жильцов, оплату за месяц (одинаковая для всех - класс Тариф), число оплативших. Реализовать метод для подсчета общей задолженности жильцов. Реализовать возможность изменения (увеличения и уменьшения) тарифа.

4. Предметная область: **Аэропорт-Билет**. В классе хранить информацию о названии аэропорта, стоимости билета (одинаковая для всех направлений - класс Билет), общее число мест во всех самолетах, число проданных билетов. Реализовать метод для подсчета общей стоимости всех проданных билетов. Реализовать возможность изменения (увеличения и уменьшения) стоимости билета.

5. Предметная область: **Банк-Вклад**. В классе хранить информацию о наименовании банка, число вкладов, размер вклада (одинаковый для всех вкладчиков - класс Вклад), размер процентной ставки. Реализовать метод для подсчета общей выплаты по процентам. Реализовать возможность изменения (увеличения и уменьшения) размера вклада.

6. Предметная область: **Отдел кадров-Норма выработки**. В классе хранить информацию о наименовании предприятия, числе работников, норму выработки часов в месяц (одинаковая для всех работников - класс Норма выработки), оплату за час, подоходный налог. Реализовать метод для подсчета общей выплаты по подоходному налогу. Реализовать возможность изменения (увеличения и уменьшения) нормы выработки.

7. Предметная область: **Фирма грузоперевозок - Тариф**. В классе хранить информацию об оплате за перевозку одной тонны грузов (не зависит от направления - класс Тариф), масса перевезенных грузов, наименование фирмы. Реализовать метод для подсчета общей выручки фирмы. Реализовать возможность изменения (увеличения и уменьшения) тарифа.

8. Предметная область: **Гостиница - Тариф**. В классе хранить информацию о названии гостиницы, числе заселенных мест, общем числе мест, оплате за день проживания (для всех жильцов одинаковая стоимость – класс Тариф). Реализовать метод для подсчета общей выручки гостиницы. Реализовать возможность изменения (увеличения и уменьшения) тарифа.

9. Предметная область: **Интернет оператор - тариф**. В классе хранить информацию о стоимости тарифа (одна для всех пользователей – класс Тариф), наименовании оператора, числе абонентов. Реализовать метод для подсчета общей выручки. Реализовать возможность изменения (увеличения и уменьшения) тарифа.

10. Предметная область: **Интернет магазин - Телевизор** по продаже телевизоров. В классе хранить информацию о стоимости телевизора (одна для всех моделей – класс Телевизор), наименовании магазина, число покупок. Реализовать метод для подсчета общей выручки. Реализовать возможность изменения (увеличения и уменьшения) стоимости телевизора.

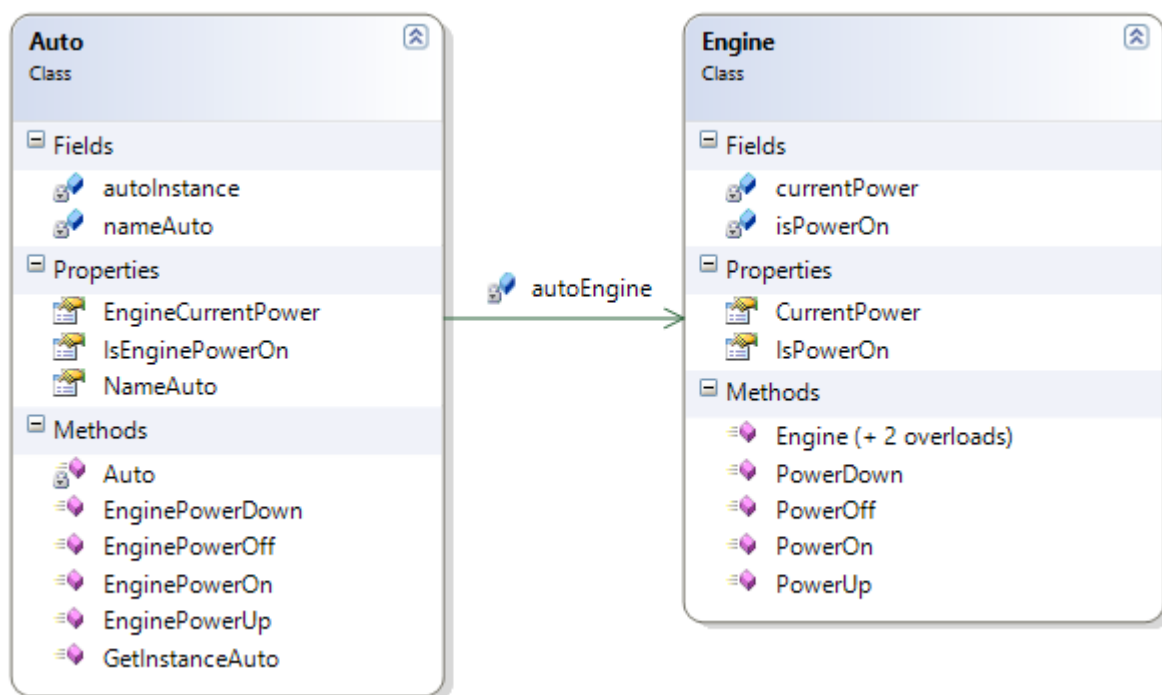
Пример выполнения работы

Пусть задана предметная область: Автомобиль – Двигатель автомобиля.

Класс Автомобиль является контейнерным классом, а класс двигатель его контейнеризируемым.

В классе Двигатель хранится информация о состоянии (включен/выключен) и о текущей мощности.

Диаграмма классов:



Класс Auto – описывает автомобиль

Поля		
autoInstance	private	Экземпляр класса Auto. Используется для реализации шаблона Singleton
nameAuto	private	Название автомобиля
autoEngine	private	Объект класса Engine (двигатель автомобиля)
Свойства		
EngineCurrentPower		Мощность двигателя. Возвращает мощность, полученную из объекта класса Engine (поле autoEngine)
IsPowerOn		Признак включения двигателя. Возвращает значение, полученное из объекта класса Engine (поле autoEngine)
NameAuto		Название автомобиля. Возвращает значение поля nameAuto
Методы		
Auto	private	Конструктор класса
EnginePowerDown EnginePowerOff EnginePowerOn		Методы управления двигателем

EnginePowerUp		
GetInstanceAuto	static	Фабричный метод для реализации шаблона Singleton. Возвращает объект класса Auto

Класс Engine – описывает двигатель

Поля		
currentPower	private	Текущая мощность двигателя
isPowerOn	private	Признак включения двигателя.
Свойства		
CurrentPower		Мощность двигателя. Возвращает значение поля currentPower
IsPowerOn		Признак включения двигателя. Возвращает значение поля isPowerOn
Методы		
Engine	private	Перегруженный конструктор: • конструктор по умолчанию • конструктор, принимающий значение текущей мощности (устанавливает признак включения IsPowerOn=true) • конструктор, принимающий значение текущей мощности и признак включения. Если двигатель выключен, то currentPower=0
PowerDown PowerOff PowerOn PowerUp		Методы управления двигателем Метод PowerOn перегружен – без параметров и с параметром начальная мощность

Вопросы для самопроверки

1. Что такое класс?
2. К какому типу относится класс (ссылочный или тип значения)?
3. Какие члены класса вы знаете?
4. Чем поле отличается от свойства?
5. Что такое конструктор класса?
6. Что такое ассоциация в ООП?
7. Чем отличается композиция от агрегации?
8. Что такое перегрузка методов?
9. Чем статический член класса отличается от экземплярного члена класса?
10. Когда выполняется статический конструктор класса?

11. Можно ли в статическом методе обращаться к нестатическим членам класса?
12. Какие модификаторы доступа вы знаете?
13. Как сделать свойство только для чтения?
14. Для чего используется ключевое слово **this** ?
15. Что такое первичный конструктор (.Net 8 и выше). Как описать класс с первичным конструктором?
16. Как вызвать деструктор класса?

Лабораторная работа №5

Использование коллекций

Цель работы:

получить навыки проектирования приложения, состоящих из нескольких взаимосвязанных классов.

Время выполнения – 4 академических часа

Задание к работе

- Для заданной предметной области спроектировать программную структуру, состоящую из 3-5 классов.
- В соответствии с разработанной диаграммой классов выполнить программную реализацию.
- Предусмотреть использование типа данных – перечисление.
- Ввод/вывод должен быть реализован вне проектируемого класса.

Примечание: меню для взаимодействия с пользователем реализовывать не нужно. Для демонстрации работы приложения достаточно инициализацию исходных данных и проверку функционала приложения реализовать в коде

Индивидуальные задания

1. Предметная область: АТС.

На АТС хранится информация о всех клиентах станции. АТС имеет список тарифов на междугородние разговоры. Клиент АТС может совершать множество звонков в различные города.

Система должна:

- позволять вводить информацию о тарифах;
- вводить информацию о клиентах и регистрировать звонки;
- по введенной фамилии о клиенте определять стоимость всех сделанных им звонков в соответствии с действующими тарифами;
- вычислять общую стоимость всех выполненных на АТС звонков.

2. Предметная область: Вокзал.

Касса вокзала имеет список тарифов на различные направления. При покупке билета регистрируются паспортные данные пассажира. Пассажир покупает билеты на различные направления.

Система должна:

- позволять вводить данные о тарифах;

- позволять вводить паспортные данные пассажира и регистрировать покупку билета;
- рассчитывать стоимость купленных пассажиром билетов;
- после ввода наименования направления, выводить список всех пассажиров, купивших на него билет;

3. Предметная область: ЖЭС.

В ЖЭС хранятся тарифы на коммунальные услуги. ЖЭС имеет информацию обо всех жильцах. При потреблении жильцами коммунальных услуг информация регистрируется в системе.

Система должна позволять выполнять следующие задачи:

- ввод тарифов
- ввод информации о жильцах и потребленных услугах;
- после ввода фамилии, выводить сумму всех потребленных услуг;
 - выводить стоимость всех оказанных услуг;

4. Предметная область: Аэропорт.

Касса аэропорта имеет список тарифов на различные направления. При покупке билета регистрируются паспортные данные.

Система должна:

- позволять вводить данные о тарифах;
- позволять вводить паспортные данные пассажира и регистрировать покупку билета;
- рассчитывать стоимость купленных пассажиром билетов;
- рассчитывать стоимость всех проданных билетов.

5. Предметная область: Банк.

Информационная система банка хранит описание процентов по различным вкладам. Система хранит информацию о вкладчиках и сделанных ими вкладах. Каждый клиент может поместить в банк только один вклад.

Система должна позволять выполнять следующие задачи:

- хранить информацию о процентах по вкладам;
- хранить информацию о клиентах;
- пополнять клиенту величину вклада;
- вычислять общую сумму выплат по процентам для всех вкладов.

6. Предметная область: Отдел расчета зарплаты.

Информационная система отдела расчета зарплаты на предприятии хранит данные о величине оплаты различных видов работ. Система хранит информацию о работниках предприятия.

Система должна позволять выполнять следующие задачи:

- вводить информацию о различных видах работ;
- вводить информацию о работниках и выполненных ими работах;
- после ввода фамилии выводить для работника зарплату;
- выводить сумму выплат всем работникам;

7. Предметная область: Фирма грузоперевозок.

Фирма имеет список тарифов по перевозке грузов. Клиент регистрируется в системе, после чего может заказать перевозку определенного объема груза.

Система должна позволять выполнять следующие задачи:

- ввод тарифов;
- регистрация клиента и заказ на перевозку грузов;
- вывод суммы заказа для определенного клиента;
- подсчет суммарной стоимости всех заказов.

8. Предметная область: Гостиница.

Информационная система гостиницы хранит информацию обо всех номерах и их стоимости. Система регистрирует клиентов. Каждый клиент может заказать один номер. При попытке заказа номера, который занят, выводится предупреждение.

Система должна позволять выполнять следующие задачи:

- ввод информации о номерах и их стоимости;
- регистрация клиента и заказ номера;
- вывод списка не занятых номеров;
- после ввода фамилии клиента, вывод стоимости проживания.

9. Предметная область: Интернет оператор.

Провайдер имеет различные тарифы доступа в Интернет за 1Мбайт в зависимости от величины абонентской платы. Информационная система провайдера хранит данные о клиентах.

Система должна позволять выполнять следующие задачи:

- ввод тарифов;
- регистрация пользователя;
- ввод данных о потребленном трафике для конкретного пользователя;
- подсчет общей стоимости реализованного трафика;
- поиск клиента, заплатившего наибольшую стоимость за услуги.

10 Предметная область: Интернет магазин.

В информационной системе хранятся данные о товарах. Клиент звонит в магазин и оставляет заказ на товар.

Система должна позволять выполнять следующие задачи:

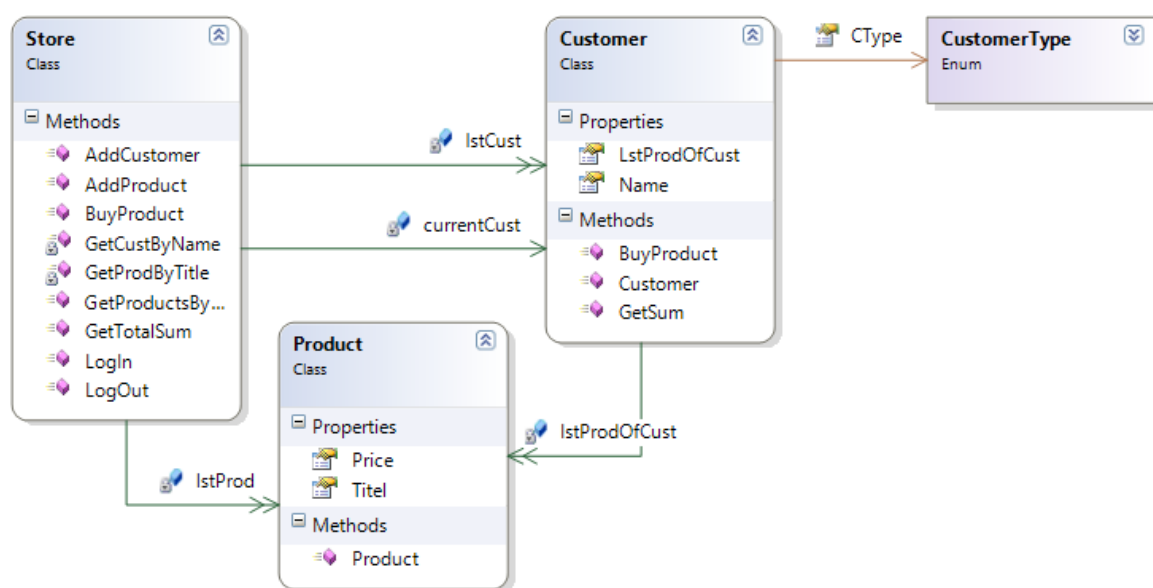
- ввод информации о товарах;
- регистрация заказа клиента на покупку определенного товара;
- после ввода фамилии покупателя вывод списка заказанных им товаров;
- после ввода фамилии покупателя вывод суммы заказа.

Пример выполнения работы

Предметная область: Торговая система.

В информационной системе хранятся данные о товарах и покупателях. Некоторые покупатели имеют статус “VIP”. Для приобретения товара вводится фамилия, после чего регистрируются купленные товары. Система должна высчитывать общую сумму проданных товаров.

Диаграмма классов:



CustomerType – перечисление. Описывает тип покупателя (VIP или Standard)

Класс Product – описывает товар

Свойства		
Price		Стоимость товара
Title		Наименование товара
Методы		

Product		Конструктор класса
---------	--	--------------------

Класс Store – описывает магазин

Поля		
lstCust	private	Список покупателей
currentCust	private	Покупатель, совершающий покупки (вошедший в систему)
lstProd	private	Список товаров
Методы		
AddCustomer		Добавление покупателя в список
AddProduct		Добавление товара в список
BuyProduct		Покупка товара текущим покупателем (вошедший в систему)
GetCustByName	private	Поиск покупателя по имени
GetProdByTitle	private	Поиск товара по названию
GetProdutsByName		Получение списка товаров, купленных покупателем с заданным именем
GetTotalSum		Получение общей стоимости всех проданных товаров
LogIn		Вход в систему (по имени) для совершения покупок
LogOut		Выход из системы (завершение покупок)

Класс Customer – описывает покупателя

Поля		
lstProd	private	Список купленных товаров
Свойства		
Name		Имя покупателя
LstProd		Список купленных товаров. Возвращает поле lstProd
CType		Тип покупателя - CustomerType
Методы		
Customer		Конструктор класса. Принимает имя покупателя и его тип
BuyProduct		Добавить товар в список покупок
GetSum		Получение общей стоимости всех купленных товаров

Код программы:

```
/// <summary>
```

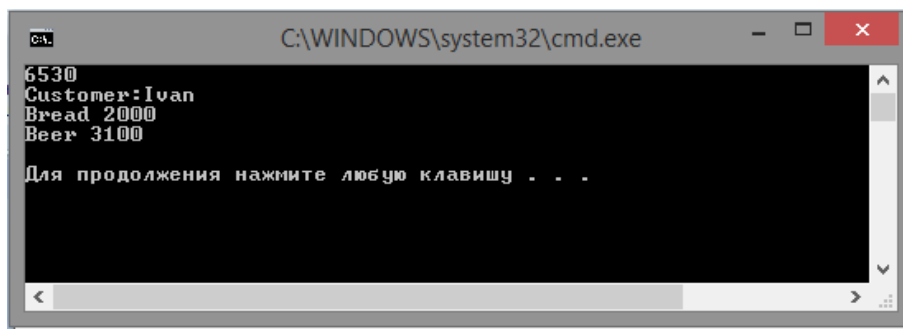
```

/// Основная программа
/// </summary>
class Program
{
    static void Main()
    {
        Store riga = new Store();
        riga.AddProduct("Milk", 1430);
        riga.AddProduct("Bread", 2000);
        riga.AddProduct("Beer", 3100);
        riga.AddCustomer("Masha", CustomerType.VIP);
        riga.AddCustomer("Ivan", CustomerType.usual);
        riga.LogIn("Ivan");
        riga.BuyProduct("Bread");
        riga.BuyProduct("Beer");
        riga.LogOut();
        riga.LogIn("Masha");
        riga.BuyProduct("Milk");
        riga.LogOut();

        Console.WriteLine(riga.GetTotalSum());
        Console.WriteLine(riga.GetProductsByName("Ivan"));
    }
}

```

Результат выполнения программы:



Вопросы для самопроверки

1. Чем коллекция **List** отличается от массива?
2. Можно ли получить доступ к элементам коллекции **List<T>** по индексу?
3. Имеется массив **names**. Что означает выражение **names[1..4]** ?
4. Имеется массив **names**. Что означает выражение **names[^2]** ?
5. Приведите пример использования коллекции **Queue**
6. Какое преимущество коллекции **Dictionary** по сравнению с **List** ?
7. Что такое **HashSet** ? Отличие **HashSet** от других коллекций.
8. Что такое **Enum** ? Приведите пример использования.
9. Как создать кортеж?
10. Как получить доступ к элементам кортежа?

Лабораторная работа №6

Наследование

Цель работы:

Изучить механизмы наследования в C#.

Время выполнения – 4 академических часа

Задание к работе

- Выделить в предметной области 2-3 варианта сущности (например, "Геометрическая фигура", "Точка", "Треугольник"), отличающиеся несколькими полями и методами. Каждый класс имеет поля, свойства и методы.
- Спроектировать UML-диаграммы классов.
- **Базовый** класс для вашей иерархии объявите **абстрактным**. Он должен содержать **абстрактные методы** и **методы с реализацией**.
- Один из наследников должен **перегружать** метод родителя.
- Один из классов должен содержать **виртуальный метод**, который **переопределяется** в одном наследнике и **не переопределяется** в другом.
- Продемонстрировать работу всех объявленных методов.
- Продемонстрировать **вызов конструктора родительского класса** при наследовании.
- Продемонстрировать вызов метода родительского класса при его **скрытии**.
- Создать класс, **закрытый для наследования**.

Примеры методов: вывод полей класса, изменение числовых полей класса (скорости, дохода, численности штата и т.д.), изменение текстовых полей класса (переименование), какая-либо динамика (полет, ремонт, переезд, открытие/закрытие и.д.)

Индивидуальные задания

1. Геометрические фигуры
2. Здания
3. Транспорт
4. Автомобили
5. Товары
6. Игровые юниты
7. Компьютерная техника
8. Игрушки
9. Мебель

10. Фильмы
11. Техника
12. Еда
13. Алкогольные напитки
14. Коктейли
15. Фрукты или овощи
16. Десерты
17. Космические объекты
18. Спорт
19. Животные
20. Растения

Вопросы для самопроверки

1. Поддерживает ли C# множественное наследование?
2. Есть базовый класс **A** и производный от него класс **B**. Будет ли вызываться конструктор класса **A** при создании объекта класса **B**?
3. Может ли абстрактный класс содержать неабстрактные члены?
4. Можно ли создать объект абстрактного класса?
5. Может ли наследник абстрактного класса тоже быть абстрактным?
6. Есть базовый класс **A** и два производных класса: **B** и **C**. Допустимо ли такое присваивание:

```
A a;  
a = new B();
```

7. Есть базовый класс **A** и два производных класса: **B** и **C**. Допустимо ли такое присваивание:

```
B b;  
b = new C();
```

8. Есть базовый класс **A** и производный класс **B**. В классе **B** добавлен метод **Execute**. Как из переменной **a** вызвать метод **Execute** :

```
A a;  
a = new B();
```

9. Есть базовый класс **A** и два производных класса: **B** и **C**. Допустимо ли такое присваивание:

```
A a;  
a = new B();  
B b = a;
```

10. Что такое «явное приведение типа»?
11. Для чего используется оператор **is** ?
12. Для чего используется оператор **as** ?
13. Как из производного класса вызвать метод базового класса?

14. Какой модификатор доступа используется для того, чтобы член класса был виден только в иерархии наследования классов?
15. **Базовый** класс имеет модификатор доступа **internal**. Может ли **производный** класс иметь модификатор доступа **public**?
16. Что такое сокрытие метода? Как реализуется?
17. Что такое переопределение метода? Как реализуется?
18. Чем отличается абстрактный метод от виртуального?

Лабораторная работа №7

Перегрузка операторов

Цель работы:

Изучить механизмы реализации полиморфизма в C#.

Задание к работе

- Спроектировать класс согласно варианту индивидуального задания. Для класса использовать отдельный модуль.
- Спроектировать конструкторы и свойства с контролем корректности вводимых значений.
- перегрузить метод `toString()`
- добавить индексирование для получения полей класса
- Перегрузить операции:
 - a) математические (имеющие смысл для объектов класса)
 - b) инкремент и декремент (изменить поля на 1)
 - c) отношения (`==`, `!=`, `<`, `>`)
 - d) `true` и `false`
 - e) преобразования типа

В методе `main()`

- Создать несколько объектов класса. Продемонстрировать использование конструкторов и свойств.
- Продемонстрировать работу всех методов и операций.

Примечание: меню для взаимодействия с пользователем реализовывать не нужно. Для демонстрации работы приложения достаточно инициализацию исходных данных и проверку функционала приложения реализовать в коде

Индивидуальные задания

1. Класс `complex` (Комплексное число). Тип полей – `double`. Перегрузить `+`, `-`, `/`, `*` для 2 комплексных, для комплексного и `double`. Сравнивать по модулю. (d) если обе части = 0, число = `false`. Преобразовать `complex` в `double` явно, назад – неявно.

2. Класс `rational` (рациональное число). Поля – `int` (числитель и знаменатель). Добавить метод нормализации (сокращение) и НОД. Перегрузить `+`, `-`, `/`, `*` для 2 `rational`, для `rational` и `int`. (d) если числитель = 0, число = `false`. Преобразовать `rational` в `int` явно, назад – неявно.

3. Класс `interval` (интервал). Поля – `double` (2 границы). Добавить метод вычисления длины. Перегрузить `+`, `-`, `*` (пересечение) – проверить возможность выполнения операции. `++` и `--` - расширение и сужение интервала.

Сравнивать по длине. (d) если длина интервала = 0, интервал = false. Преобразовать интервал в double явно (длина), назад – также явно.

4. Класс квадратное уравнение. Поля – int (a, b, c). Добавить метод нахождения корней. Перегрузить +, -, ++, --, * на число, / на число. Сравнить на == и !=. (d) если уравнение не имеет корней, уравнение = false. Преобразовать в число (a) и назад (ax^2) – в обоих случаях явно.

5. Класс полином $ax+by+cz$. Поля – int (a, b, c). Перегрузить +, -, ++, --, * на число, / на число. Сравнить на == и !=. (d) если $a=b=c=0$, полином = false. Преобразовать в число (a) и назад (ax) – в обоих случаях явно.

6. Класс vector. Поля – int (a, b, c). Перегрузить +, -, ++, --, *, * на число, / на число. Сравнить на == и !=. (d) если $a=b=c=0$, вектор = false. Преобразовать в число (модуль) и назад (ax) – в обоих случаях явно.

7. Класс квадратная матрица 2*2 типа int. К полям обращаться через индексатор. Перегрузить +, -, ++, --, *, * на число, / на число. Сравнить на == и !=. (d) если определитель = 0, матрица = false. Преобразовать в число (опредетитель) и назад ($a,0,0,a$) – в обоих случаях явно.

8. Класс – множество символов. В качестве поля – массив символов. Добавить метод, проверяющий корректность множества (нет повторений) – нужен сеттеру. Перегрузить +, -, * (пересечение). Сравнить на == и !=. (d) если множество пустое, оно false. Преобразовать в String – неявно, назад – явно.

9. Класс – окружность. Задается координатами центра и радиусом (double). Добавить методы вычисления периметра и площади. Перегрузить ++, --, -, *, / - изменяя соответственно радиус. Сравнивать окружности по площади. (d) если центр в начале координат, окружность = false. Преобразовать double в окружность – с центром в начале координат и радиусом double (неявно). Назад – явно.

10. Класс – треугольник. Задается длинами сторон (double). Добавить методы вычисления периметра и площади. Перегрузить ++, --, -, *, / - изменяя соответственно все 3 стороны. Сравнивать по площади. (d) если равносторонний = false. Преобразовать double в треугольник – равносторонний (неявно). Назад – явно.

Вопросы для самопроверки

1. Для чего используется перегрузка операторов?
2. Какие операторы перегружаются обязательно попарно?
3. Для чего используется оператор **Implicit** ?
4. Для чего используется оператор **Explicit** ?
5. То такое индексатор?
6. Методы перегрузки операторов являются статическими или экземпляльными?
7. Что такое «метод расширения (расширяющий метод, extension method)?

Лабораторная работа №8 Полиморфизм

Цель работы:

Изучить механизмы реализации полиморфизма в C#.

Ознакомиться с основными подходами при использовании интерфейсов.

Время выполнения – 4 академических часа

Задание к работе

- Составить диаграмму классов проектируемой системы.
- Запрограммировать классы в соответствии с новой диаграммой.
- Проиллюстрировать использование интерфейсов.
- Показать вызов метода интерфейса через интерфейсную ссылку.
- Применить в программе шаблон проектирования Strategy.

Примечание: меню для взаимодействия с пользователем реализовывать не нужно. Для демонстрации работы приложения достаточно инициализацию исходных данных и проверку функционала приложения реализовать в коде

Индивидуальные задания

1. Предметная область: АТС. АТС имеет список тарифов на междугородние разговоры. Есть два типа тарифов: обычный и льготный. В классе АТС реализовать методы добавления обычного тарифа и добавления льготного тарифа. Класс АТС должен выполнять вычисление средней стоимости тарифов с учетом скидки.

2. Предметная область: Вокзал. Класс “вокзал” имеет список тарифов на различные направления. На некоторые тарифы может быть предоставлена скидка, заданная в процентах. В классе “вокзал” реализовать методы добавления нового тарифа со скидкой и без скидки, поиск направления с минимальной стоимостью.

3. Предметная область: ЖЭС. ЖЭС имеет информацию обо всех жильцах. Имеется два типа жильцов со льготами и без льгот. В классе ЖЭС реализовать метод добавления нового жильца имеющего льготы и без льгот, а так же метод подсчета стоимости всех оказанных услуг.

4. Предметная область: Аэропорт. Касса аэропорта имеет список тарифов на различные направления. Тариф содержит название направления и стоимость перевозки. На некоторые направления предоставляется фиксированная скидка. В классе аэропорт реализовать метод добавления нового тарифа и метод поиска направления с максимальной стоимостью.

5.Предметная область: Банк. Система хранит информацию о вкладчиках и сделанных ими вкладах. Класс “вкладчик” содержит имя вкладчика и величину вклада. Некоторым вкладчикам при создании вклада на счет может дополнительно перечисляться фиксированная сумма. В классе банк реализовать методы добавления нового вкладчика и метод вычисления общей суммы вкладов.

6. Предметная область: Отдел расчета зарплаты. Информационная система отдела расчета зарплаты на предприятии хранит данные о величине оплаты за различные виды работ. На некоторые виды работ предоставляется надбавка, заданная в процентах. В классе “отдел расчета зарплаты” реализовать методы добавления нового типа работ и метод вычисления средней величины оплаты.

7.Предметная область: Фирма грузоперевозок. Фирма имеет список тарифов по перевозке грузов. Класс тариф хранит наименование тарифа и цену. На не которые тарифы предоставлена скидка, заданная в процентах. В классе фирма реализовать методы добавления нового тарифа и метод поиска тарифа с минимальной стоимостью.

8.Предметная область: Гостиница. Информационная система гостиницы хранит информацию обо всех номер ах и их стоимости. На проживание в некоторых номерах предоставляется скидка, заданная в процентах. В классе “гостиница” реализовать метод добавления информации о номере и метод вычисления средней стоимости.

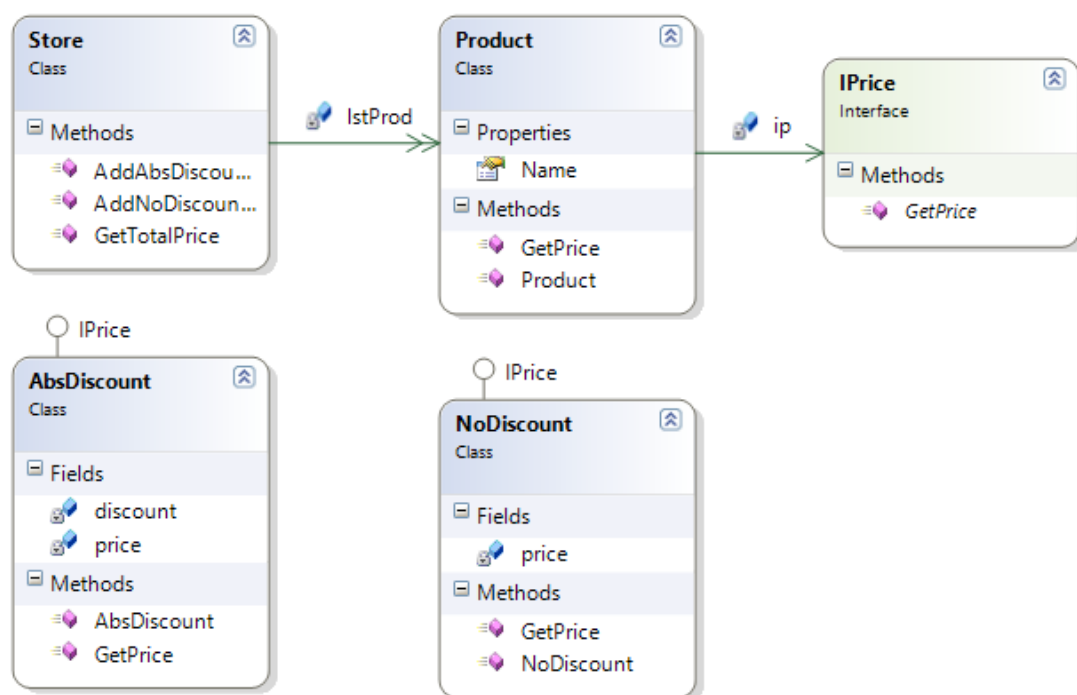
9. Предметная область: Интернет оператор. Информационная система провайдера хранит данные о клиентах. Некоторым клиентам предоставляется фиксированная скидка. В классе “оператор” реализовать метод добавления нового клиента и метод вычисления суммарной стоимости оказанных услуг.

10 Предметная область: Интернет магазин. В информационной системе хранятся данные о товарах. Класс «товар» содержит стоимость товара и его наименование. На некоторые товары предоставляется скидка, заданная в процентах. В классе магазин реализовать метод добавления нового товара имеющего скидку и без скидки, также метод поиска товара с минимальной стоимостью.

Пример выполнения работы

Пусть есть магазин с множеством продуктов. На некоторые продукты предоставляется скидка. Требуется подсчитать суммарную стоимость товаров в магазине с учетом предоставляемых скидок.

UML диаграмма классов:



Интерфейс IPrice – описывает метод получения стоимости товара

Методы		
GetPrice		Возвращает стоимость товара

Класс Product – описывает товар

Поля		
ip	private	Объект класса, реализующего интерфейс IPrice
Свойства		
Name		Название товара
Методы		
Product		Конструктор класса.
GetPrice		Возвращает стоимость товара

Класс AbsDiscount – реализация интерфейса IPrice для подсчета стоимости товара со скидкой

Поля		
discount	private	Величина скидки
Price	private	Стоимость товара
Методы		
AbsDiscount		Конструктор класса. Принимает значения стоимости товара и скидки

GetPrice		Реализация интерфейса IPrice для подсчета стоимости товара со скидкой
----------	--	---

Класс NoDiscount – реализация интерфейса IPrice для подсчета стоимости товара без скидки

Поля		
Price	private	Стоимость товара
Методы		
AbsDiscount		Конструктор класса. Принимает значение стоимости товара
GetPrice		Реализация интерфейса IPrice для подсчета стоимости товара без скидки

Класс Store – описывает магазин

Поля		
LstProd	private	Список товаров
Методы		
AddAbsDiscount		Добавление товара со скидкой
AddNoDiscount		Добавление товара без скидки
GetTotalPrice		Получение общей стоимости всех товаров

Код программы

```
class Program
{
    static void Main(string[] args)
    {
        Store riga = new Store();
        riga.AddAbsDiscountProduct("bread", 3200, 200);
        riga.AddNoDiscountProduct("milk", 1200);
        Console.WriteLine(riga.GetTotalPrice());
    }
}
```

Вопросы для самопроверки

1. Что такое «Полиморфизм» в ООП?
2. Допускается ли в С# множественное наследование интерфейсов?

3. Может ли класс, реализующий интерфейс, реализовать только часть методов, описанных в интерфейсе?
4. Может ли интерфейс содержать методы с модификатором **private**?
5. Может ли интерфейс содержать статические методы
6. Как описать конструктор интерфейса?
7. Может ли интерфейс содержать реализация методов и свойств по умолчанию?
8. Что такое «**коллизия имен**»? Когда она возникает?
9. Что такое «**склеивание**» при решении коллизии имен?
10. Что такое «**явная реализация интерфейса (кастинг)**» при решении коллизии имен?
11. Как получить доступ к методу интерфейса при кастинге?
12. Чем отличается интерфейс от абстрактного класса?
13. Для чего используется интерфейс **Comparable**?

Лабораторная работа №9

Множественное наследование

Цель работы:

получить навыки проектирования приложения, состоящих из нескольких взаимосвязанных классов и интерфейсов

Время выполнения – 8 академических часов

Задание к работе

1. Описать семейство классов, имеющих общий функционал (один базовый абстрактный класс и несколько производных классов), при этом в каждом наследнике присутствует дополнительно свой функционал. Набор дополнительных функций в производных классах может быть произвольным.

2. Дополнительный функционал описать в виде набора **интерфейсов**

3. Одно из общих свойств описать в виде **перечисления**

4. Одна из **общих** функций должна быть реализована по-своему в каждом производном классе.

5. Одна из **общих** функций также должна быть реализована в производных классах (например, изменение скорости, использование оружия, доставка груза). При этом должно быть несколько вариантов реализации (несколько классов), например, персонажам игры доступны разные инструменты – каждый инструмент может использоваться разными персонажами. Значит, возможны разные варианты общей функции – «Использовать инструмент». Конкретный вариант реализации выбирается **при создании объекта** (применить шаблон проектирования «Мост» («Bridge»)).

6. Для создания объектов использовать шаблон проектирования «Абстрактная фабрика» (Abstract factory) или «Построитель» (Builder)

7. В классе Program создать коллекцию разных объектов (см. п.1). Затем для каждого элемента коллекции вызвать **все методы, доступные** для данного объекта.

Примечание: НЕ НУЖНО разрабатывать реальную реализацию функций. Достаточно вывести в консоль сообщение о том, какое действие выполняется, и кем.

Примечание: меню для взаимодействия с пользователем реализовывать не нужно. Для демонстрации работы приложения достаточно инициализацию исходных данных и проверку функционала приложения реализовать в коде

Примечание: для вызова **всех** доступных методов, необходимо проверить, реализует ли объект соответствующий интерфейс

Индивидуальные задания

1. Предметная область: Телевизор.

- Пример общих свойств: наименование, тип экрана (например: плазма, OLED ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Show (варианты описаны в других классах, примеры: эфир, кабель, HDMI ...)
- Примеры дополнительных функций: Интернет-браузер, Распознавание движений, Голосовой помощник ...

2. Предметная область: Строительная бригада.

- Пример общих свойств: наименование, тип (организация, ИП, частное лицо ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), BuildWalls (варианты описаны в других классах, примеры: из кирпича, из пеноблоков, монолитные ...)
- Примеры дополнительных функций: поставка стройматериалов, разработка проекта, консультация заказчика ...

3. Предметная область: Риелтор.

- Пример общих свойств: имя, тип недвижимости
- Общие функции: GetInfo (в каждом классе реализуется по-своему), DescribeObject (варианты описаны в других классах, примеры: по E-mail, по телефону, через Viber ...)
- Примеры дополнительных функций: выезд на объект, подготовка документов, проверка банкнот ...

4. Предметная область: Кухонный процессор.

- Пример общих свойств: название, тип(настольный, ручной, встроенный ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Process (варианты описаны в других классах, примеры: шинкование, измельчение, мясорубка ...)
- Примеры дополнительных функций: миксер, блендер, тестомешалка ...

5. Предметная область: Фирма доставки товаров.

- Пример общих свойств: название, тип(городской, междугородний, международный ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Deliver (варианты описаны в других классах, примеры: автомобилем, DHL, AutoLight Express ...)

- Примеры дополнительных функций: сопровождение груза, страхование, отслеживание этапов доставки ...

6. Предметная область: Пиццерия.

- Пример общих свойств: название, размер (маленькая, средняя, большая ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Bake (варианты описаны в других классах, примеры: на толстой основе, на тонкой основе, закрытая)
- Примеры дополнительных функций: (добавки) острая, с голубым сыром, на углях...

7. Предметная область: Медперсонал.

- Пример общих свойств: имя, профиль (хирург, терапевт, медсестра ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Diagnose (варианты описаны в других классах, примеры: осмотр, МРТ, рентген...)
- Примеры дополнительных функций: сделать укол, выписать больничный, сделать перевязку ...

8. Предметная область: Гостиница.

- Пример общих свойств: название, тип (гостиница, хостел, агроусадьба ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Pay (варианты описаны в других классах, примеры: наличные, карта, банковский перевод ...)
- Примеры дополнительных функций: трансфер от аэропорта, минибар, бассейн, интернет ...

9. Предметная область: Автомобиль.

- Пример общих свойств: модель, тип (легковой, грузовой ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), ChangeGear (варианты описаны в других классах и зависит от коробки передач, примеры: механика, автомат, вариатор...)
- Примеры дополнительных функций: передвижение, перевозка груза, перевозка пассажиров ...

10 Предметная область: Принтер.

- Пример общих свойств: модель, формат печати (A4, A3, 44” ...)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Print (варианты описаны в других классах и зависит от технологии печати, примеры: лазерная, струйная, сублимационная...)

- Примеры дополнительных функций: печать на листах, печать с рулона, сортировка, web-интерфейс ...

11 Предметная область: Путешествия

- Пример общих свойств: маршрут, вид (индивидуальный, групповой, семейный)
- Общие функции: GetInfo (в каждом классе реализуется по-своему), Travel (варианты описаны в других классах и зависит от способа путешествия, примеры: пешком, на велосипеде, на автобусе...)
- Примеры дополнительных функций: посещение достопримечательностей, мангал, прыжок с тарзанки ...

Пример выполнения работы

Предметная область: Пошаговая стратегия.

Семейство объектов – персонажи игры: Peasant и Necromancer

Общие свойства: Name (имя) и Gender (пол)

Пол описан как перечисление:

```
enum Gender
{
    Male, Female
}
```

Дополнительные свойства:

Возможность перемещения. Описано в интерфейсе IMovable. Реализуется в классе Peasant.

```
internal interface IMovable
{
    void Walk();
    void Run();
}
```

Выполнение работы. Описано в интерфейсе IWorker. Реализуется в классе Peasant.

```
internal interface IWorker
{
    void DoWork();
}
```

Наложение заклятья. Описано в интерфейсе IWizard. Реализуется в классе Necromancer.

```
internal interface IWizard
{
    void Spell();
}
```

Пример реализации метода DoWork:

```
public void DoWork()
{
    Say("I work on my garden");
}
```

Общий метод GetInfo().

В классе Peasant реализуется так:

```
public override void GetInfo()
{
    Say($"I am peasant, my gender is {Gender}");
}
```

В классе Necromancer реализуется так:

```
public override void GetInfo()
{
    Say($"I am necromancer, my gender is {Gender}");
}
```

Метод Say:

```
protected void Say(string message)
{
    Console.WriteLine($"{Name} says: {message}");
}
```

Общий метод UseWeapon – использовать оружие

Предлагаемые виды оружия: Longbow, Sword и Poleaxe

Оружие реализует интерфейс IWeapon:

```
internal interface IWeapon
{
    void Attack();
}
```

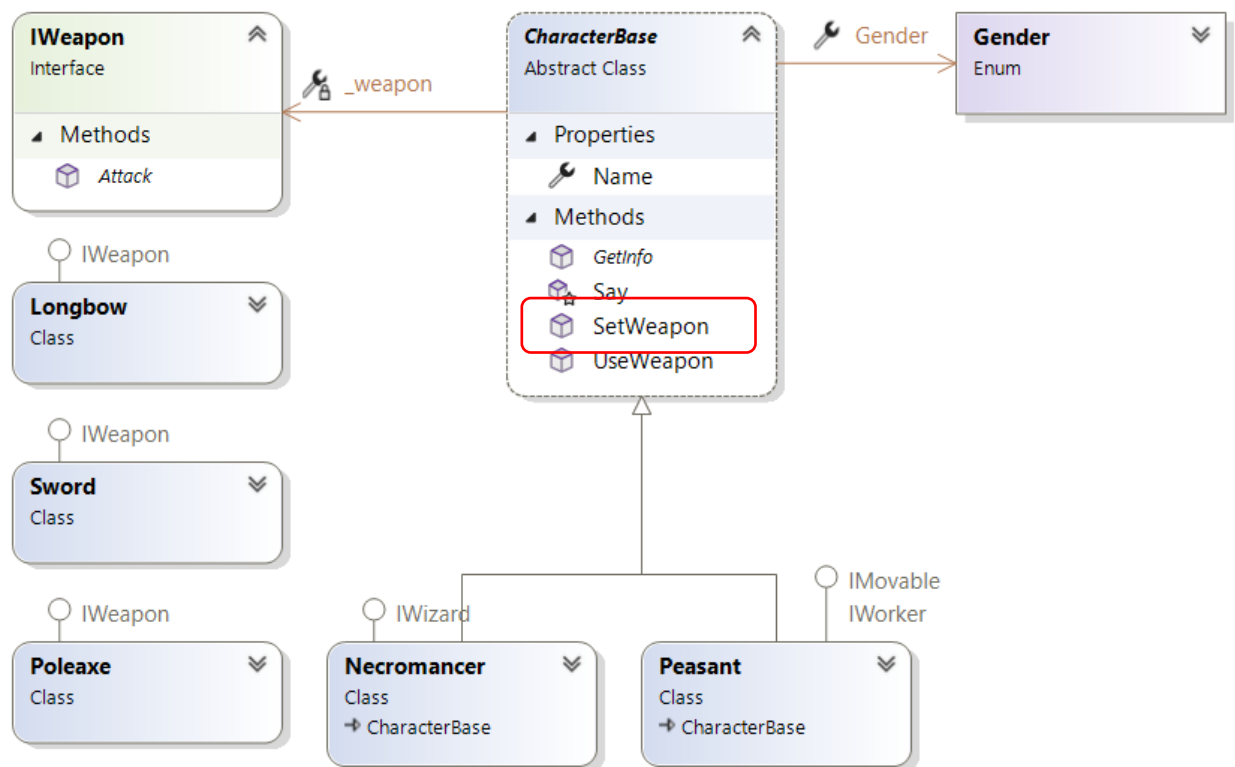
Пример реализации в классе Longbow:

```
public void Attack()
{
    Console.WriteLine("An arrow shot from a bow");
}
```

Шаблон проектирования «Мост»

В качестве абстракции (моста) выделим абстрактный класс CharacterBase, описывающий общие члены классов персонажей, и взаимодействующий с оружием через интерфейс IWeapon.

Диаграмма классов:



Теперь с помощью метода **SetWeapon** можно любому персонажу назначить любое оружие.

Пример создания объекта **Peasant**:

```
var peasant = new Peasant {Name="Bob", Gender=Gender.Male};
peasant.SetWeapon(new Sword());
```

Создание объектов с помощью шаблона Строитель

Пример абстрактного класса строителя:

```
internal abstract class AbstractBuilder
{
    protected string name = String.Empty;
    protected Gender gender;
    protected IWeapon? weapon;
    public AbstractBuilder SetName(string name)
    { this.name = name; return this; }
    public AbstractBuilder SetGender(Gender gender)
    { this.gender = gender; return this; }
    public AbstractBuilder SetWeapon(IWeapon weapon)
    { this.weapon = weapon; return this; }

    public abstract CharacterBase Build();
}
```

```
}
```

Метод Build() будет реализован в наследниках PeasantBuilder и NecromancerBuilder, например:

```
public override CharacterBase Build()
{
    var peasant = new Peasant {Name=name, Gender=gender};
    if(weapon!=null)
        peasant.SetWeapon(weapon);
    return peasant;
}
```

Класс Director с помощью строителей будет создавать конкретные объекты классов, например:

```
internal static class Director
{
    /// <summary>
    /// Создать невооруженного мужчину
    /// </summary>
    /// <param name="name">Имя персонажа</param>
    /// <param name="builder">объект строителя</param>
    /// <returns></returns>
    public static CharacterBase GetMale(string name, AbstractBuilder builder)
    {
        }

    /// <summary>
    /// Создать невооруженную женщину
    /// </summary>
    /// <param name="name">Имя персонажа</param>
    /// <param name="builder">объект строителя</param>
    /// <returns></returns>
    public static CharacterBase GetFemale(string name, AbstractBuilder
builder)
    {
        }

    /// <summary>
    /// Создать женщину-лучника
    /// </summary>
    /// <param name="name">Имя персонажа</param>
    /// <param name="builder">объект строителя</param>
    /// <returns></returns>
    public static CharacterBase GetArcherFemale(string name, AbstractBuilder
builder)
    {
        return builder.SetName(name)
            .SetGender(Gender.Female)
            .SetWeapon(new Longbow())
            .Build();
    }
}
```

```

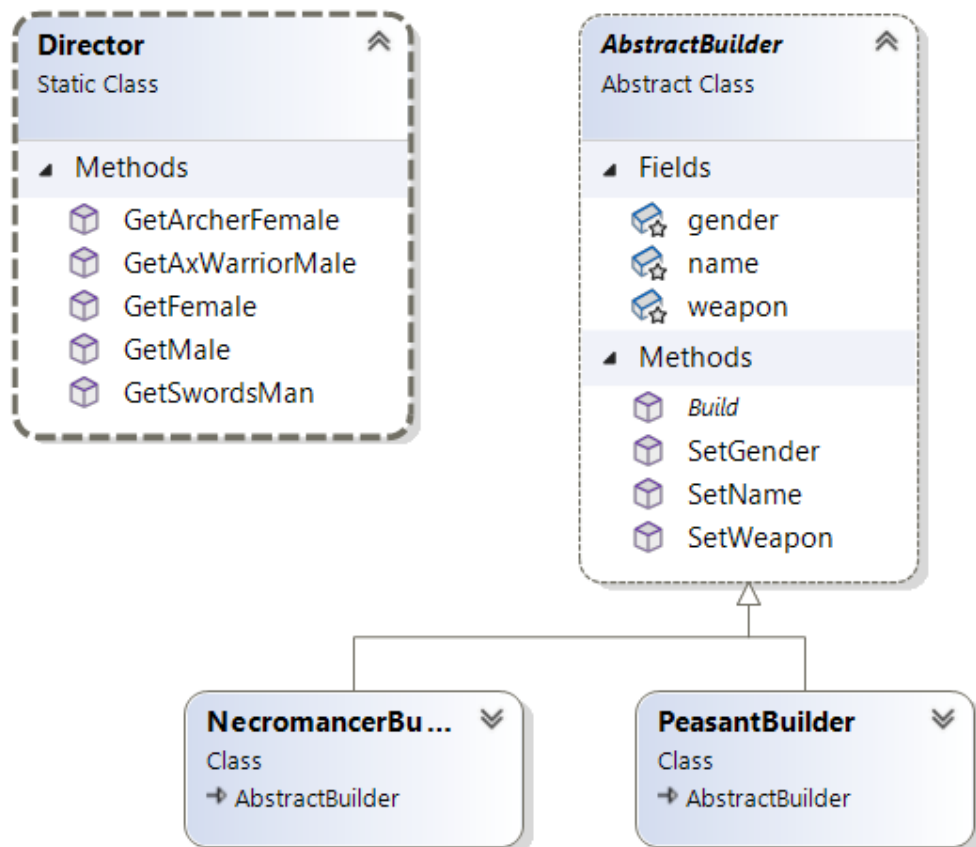
    }

    /// <summary>
    /// Создать мечника
    /// </summary>
    /// <param name="name">Имя персонажа</param>
    /// <param name="builder">объект строителя</param>
    /// <returns></returns>
    public static CharacterBase GetSwordsMan(string name, AbstractBuilder
builder)
    {
        }

    /// <summary>
    /// Создать мужчину, вооруженного топором
    /// </summary>
    /// <param name="name">Имя персонажа</param>
    /// <param name="builder">объект строителя</param>
    /// <returns></returns>
    public static CharacterBase GetAxWarriorMale(string name, AbstractBuilder
builder)
    {
        }
}

```

Диаграмма классов:



Пример создания объектов в классе Program:

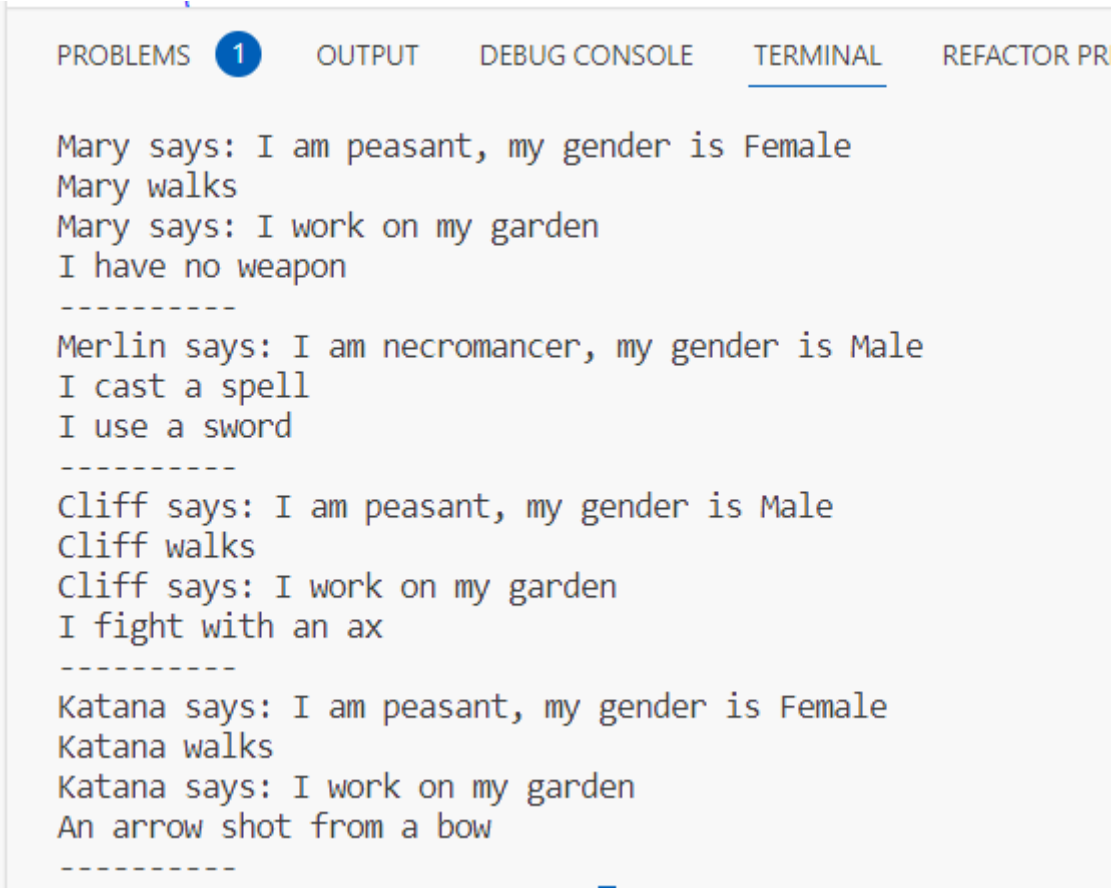
```
var characters = new List<CharacterBase>();

var peasantBuilder = new PeasantBuilder();
var necromancerBuilder = new NecromancerBuilder();

characters.AddRange(new CharacterBase[]
{
    Director.GetFemale("Mary", peasantBuilder),
    Director.GetSwordsMan("Merlin", necromancerBuilder),
    Director.GetAxWarriorMale("Cliff", peasantBuilder),
    Director.GetArcherFemale("Katana", peasantBuilder)
});
```

Далее в цикле нужно обойти коллекцию **characters** и вызвать **Все** доступные методы

Пример результата работы программы:



The screenshot shows a terminal window with a tabbed interface. The 'TERMINAL' tab is selected. The output text is as follows:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL REFACTOR PR

Mary says: I am peasant, my gender is Female
Mary walks
Mary says: I work on my garden
I have no weapon
-----
Merlin says: I am necromancer, my gender is Male
I cast a spell
I use a sword
-----
Cliff says: I am peasant, my gender is Male
Cliff walks
Cliff says: I work on my garden
I fight with an ax
-----
Katana says: I am peasant, my gender is Female
Katana walks
Katana says: I work on my garden
An arrow shot from a bow
-----
```